

[🏠](#) / [Мое](#) / [Мои проекты](#) / AI-агент службы поддержки

Проект: AI-агент службы поддержки

15 августа
2026 г.

> _ НЕДОСТ...

📄 НЕДОСТ...

○ В ПРОЦ...

[Репозиторий](#)[Помощь](#)

Подготовка



1



2



3



4



5



6



7



8



9

> 10



Проверка

Подготовка окружения

Поднимаем рабочее окружение: репозиторий проекта, утилиты командной строки, облачный каталог и serverless-базу YDB. После этого шага у вас есть чистая площадка, на которой дальше разворачиваются все компоненты.

В проекте используется Yandex Cloud — поэтому кроме git и редактора нам понадобится `yc` (Yandex Cloud CLI) и доступ к консоли. Всё, что мы деплоим дальше (Cloud Functions, Workflows, MCP-шлюзы), создаётся через `yc` либо через UI AI Studio. Обращения пользователей и история диалогов хранятся в YDB Serverless — это дешевле на старте (платим за запросы, не за простой).

Ссылки

- [Инструкция по установке yc](#)
- [Yandex Managed Service for YDB](#) — описание сервиса в Yandex Cloud

Задачи

1. Склонируйте созданный на GitHub репозиторий себе локально.
2. Установите утилиту `yc`.
3. Инициализируйте профиль: `yc init` → выберите облако и каталог (или создайте новый).



Все курсы

установите `gunicorn`. Он нужен для CLI `yandex-ai-studio` из пакета `yandex-ai-studio-sak` (шаг про RAG). Для Python-варианта пригодится и пакет `ydb` (инициализация схемы — шаг про MCP и YDB). Ставьте в удобное вам окружение (`venv`, `uv`, `conda` — на ваш выбор).

5. Создайте базу YDB Serverless в каталоге:

```
1 yc ydb database create help-desk-db --serverless
```



Создание идёт асинхронно (~40 секунд). Дождитесь статуса `RUNNING`: `yc ydb database get help-desk-db --format json | jq -r .status` (значения: `PROVISIONING` → `RUNNING`).

6. Получите и запишите значения в `.env` (реальный файл, он в `.gitignore`):

```
1 ENDPOINT=$(yc ydb database get help-desk-db --format json | jq -r .endpoint)
2 YDB_ENDPOINT=$(echo "$ENDPOINT" | sed -E 's|(grpc://[^/?]+).*|\1|')
3 YDB_DATABASE=$(echo "$ENDPOINT" | sed -E 's|.*database=([^&]+).*|\1|')
4 echo "YDB_ENDPOINT=$YDB_ENDPOINT"
5 echo "YDB_DATABASE=$YDB_DATABASE"
```



У `yc ydb database get` нет отдельного поля `.database` — путь зашит в `endpoint` как query-параметр `?database=/ru-central1/...`. Поэтому парсим `sed`'ом. Поле `.name` возвращает короткое имя `help-desk-db`, для SDK не подходит. Дальше значения перенесем в `Lockbox`.

7. Создайте файл `.env.example` в репозитории — те же переменные без значений (шаблон для проверяющего). Реальные значения держите в `.env`, который добавлен в `.gitignore`.

Результат шага

- В консоли `yc config list` показывает `cloud-id` и `folder-id`.
- В репозитории есть `README.md`, `.env.example`, `.gitignore` (включая `.env`).
- `yc serverless function list` не падает с ошибкой авторизации.
- `yc ydb database get help-desk-db` возвращает статус базы `RUNNING` (не `HEALTHY`); `YDB_ENDPOINT` и `YDB_DATABASE` записаны в `.env`.

Подсказки

- Узнать `folder-id` можно с помощью CLI:

```
1 yc resource-manager folder list
```

